



西南交通大学
Southwest Jiaotong University

《视觉计算》课程设计报告

设计题目: 云计算技术课程设计实验报告

组 号: 2023112621,2023114337

成员姓名: 程泽凯, 王晨羽

班 级: 2023-04 班

指导教师: 戴朋林、丛荣 老师

2026 年春季学期

《视觉计算》课程设计任务书

设计 题目	云计算技术课程实验报告	成绩
课 程 设 计 主 要 内 容	针对计算机视觉领域某一特定问题，例如：目标检测、人脸识别、图像分割、图像生成等，查阅国内外近 5 年来相关技术资料，选择 3 篇以上具有代表性论文进行全文阅读，分析并理解其原理和方法。撰写一篇不少于 4000 字的课程设计报告，对相关论文的理论原理及算法进行阐述，对其中的方法动手搭建模型（建议基于 MindSpore），编写程序复现实验结果，进行对比分析，给出结论。鼓励提出一定的改良措施，完成实验（建议基于华为 MindSpore）。 课程设计以组队的形式开展，每组 2-3 人，自行组队，报告中明确每个人的分工。报告使用老师提供的 latex 模版，编译方式选 XeLatex。模版已写好第一至第六章的目录，尽量按照该目录撰写报告。	
指 导 教 师 评 语	建议：从学生的工作态度、工作量、设计（论文）的创造性、学术性、实用性及书面表达能力等方面给出评价。 签名： 年 月 日	

目录

一、学生信息与分工	1
二、实验目标与总体方案	1
三、华为云环境信息	2
四、第一部分：云计算平台搭建	3
4.1 任务 1：应用容器化	3
4.2 任务 2：CCE 集群搭建	4
4.3 任务 3：应用部署	5
4.4 任务 4：Redis 持久化存储	7
4.5 任务 5：ConfigMap Volume 挂载	8
4.6 任务 6：HPA 弹性伸缩	11
五、第二部分：Spark 大数据分析	14
5.1 A-0 环境部署	14
5.2 A-1 数据清洗	15
5.3 A-2 Spark SQL 统计分析	15
5.4 A-3 性能对比与 Amdahl 分析	16
六、关键文件与代码说明	17
七、总结与收获	17
八、评分要求对照	20
8.1 第一部分评分对照	20
8.2 第二部分评分对照	20
8.3 报告质量要求	20
附录 A 核心命令清单	20
附录 B 截图清单	20

一、学生信息与分工

表 1 课程设计基本信息

项目	内容
课程名称	云计算技术
姓名	程泽凯，王晨羽
学号	2023112621,2023114337
班级	2023-04 班
完成方式	个人完成，可按实际组队情况修改
主要方向	CCE 容器平台搭建与 Spark 大数据分析

本课程设计围绕“本地容器化开发、华为云 Kubernetes 部署、并行计算作业运行和实验报告整理”展开。本人主要完成 Flask 后端 API、Nginx 前端页面、Redis 数据库联调、Docker 镜像构建、CCE YAML 编写、Redis 持久化验证、HPA 弹性伸缩验证以及 Spark 数据分析代码与性能对比分析。

二、实验目标与总体方案

本课程设计的目标是在华为云平台上完成一套可运行、可验证、可展示的云原生应用与并行计算实验。第一部分基于华为云 CCE 搭建 Kubernetes 平台，部署“Flask 后端 API + Redis 数据库 + Nginx 前端页面”的两层 Web 应用，并完成镜像仓库、对外暴露、配置分离、数据持久化和弹性伸缩验证。第二部分选择 Spark 大数据分析方向，在同一 CCE 集群上通过 Spark Operator 运行 PySpark 作业，完成数据清洗、统计分析和性能对比。

实验设计遵循从简单到复杂的顺序：首先在本地用 Docker Compose 验证应用功能，确保后端接口、前端页面和 Redis 访问没有逻辑错误；然后将镜像推送到 SWR，使云端集群可以拉取同一版本镜像；再使用 CCE 部署 Kubernetes 资源，通过 Deployment 管理副本，通过 Service 暴露服务，通过 ConfigMap 和 Secret 管理配置，通过 PVC 保存 Redis 数据；最后使用 HPA 观察负载变化下的自动扩缩容。这样的顺序可以减少排错范围，避免把应用代码问题、镜像问题和集群部署问题混在一起。

整体技术路线如表2所示。

表 2 课程设计总体技术路线

阶段	主要工作
本地开发	编写 Flask API、Nginx 静态页、Redis 访问逻辑和 Docker Compose 联调配置。
镜像管理	构建 backend/frontend 镜像，推送至华为云 SWR 私有镜像仓库。
CCE 部署	在 CCE 集群中部署 Deployment、Service、ConfigMap、Secret、PVC 和 HPA。
持久化与配置	使用 EVS PVC 保存 Redis 数据，使用 ConfigMap Volume 挂载 Nginx 配置文件。
并行计算	使用 Spark Operator 提交 PySpark 作业，完成清洗、SQL 查询和性能对比。
报告整理	整理关键命令、截图证据、实验结果、问题分析和总结。

三、华为云环境信息

本实验涉及的华为云服务包括 SWR、CCE、ELB、OBS 和 EVS。SWR 用于存储本地构建的 Docker 镜像；CCE 用于运行 Kubernetes 工作负载；ELB 由 LoadBalancer 类型 Service 自动创建，用于对外暴露后端 API；OBS 用于存放 Spark 分析数据集；EVS 云硬盘通过 PVC 方式为 Redis 提供持久化存储。

表 3 华为云服务与用途

服务	在本实验中的作用
SWR	保存 backend 和 frontend 镜像，CCE 部署时从 SWR 拉取。
CCE	托管 Kubernetes 集群，运行 Pod、Deployment、Service、PVC 和 HPA。
ELB	为 LoadBalancer Service 提供公网访问入口。
OBS	存放 Spark 待分析数据集，通过 s3a 协议读取。
EVS	为 Redis PVC 提供云硬盘持久化能力。

实验环境截图应包含以下信息：Region、CCE 集群版本、Worker 节点规格、节点状态和 Kubernetes 版本。节点验证命令如下：

本次实验使用的主要本地目录为 cloud-computing-final-project/ 和 cloud-report-tex/。前者保存 Dockerfile、Compose 文件、Kubernetes YAML、Spark 脚本和辅助数据；后者用于整理实验报告和截图。云端 Region 使用华为云华南区域，SWR 组织名为 czk-course。报告截图均尽量保留命令行提示符、资源名称、状态

列和返回结果，便于对应评分细则检查。

```
kubectl get nodes -o wide
```

截图要求为所有 Worker 节点状态均为 Ready，并且输出中包含 VERSION 列。

四、第一部分：云计算平台搭建

4.1 任务 1：应用容器化

本任务从本地应用开始实施。实验中先编写 Flask 后端服务，提供 `/api/ping` 健康检查接口和 `/api/visit` Redis 写入接口；随后编写 Nginx 前端首页，在页面中展示程泽凯、王晨羽及学号 2023112621,2023114337，用于验收时识别。完成代码后，使用 Docker Compose 同时启动 backend、frontend 和 Redis，先在本地确认接口、页面和 Redis 访问逻辑均正常，再进入镜像构建和推送环节。

后端 `requirements.txt` 中除 Flask 和 Redis 客户端外，额外加入 `requests` 包，满足“加入至少 1 个自选 Python 包”的要求。后端 Dockerfile 保留 `builder/runtime` 多阶段构建结构，并通过 `PIP_INDEX_URL` 默认使用清华 PyPI 镜像源；前端 Dockerfile 使用 Nginx 镜像托管静态首页。构建镜像时 Docker Desktop 曾出现 BuildKit 服务异常，处理方式是重启 Docker Desktop、执行 `ws1 --shutdown` 并清理构建缓存。推送 SWR 时还遇到 BuildKit provenance manifest 无法被 SWR 解析的问题，因此重新使用 `docker build --provenance=false` 构建镜像，再执行 `docker tag` 和 `docker push`。最终 SWR 组织 `czk-course` 下出现 `backend` 与 `frontend` 两个镜像，Tag 均为 `v1`。

多阶段构建的作用是将依赖安装和运行环境分离，减少最终镜像中不必要的构建缓存和临时文件。后端镜像在 `builder` 阶段安装 Python 依赖，在 `runtime` 阶段只复制运行所需文件；前端镜像则只保留 Nginx 与静态页面。这样做既符合课程要求，也更接近实际云端交付流程。Docker Compose 联调阶段，我重点观察了两个现象：一是 `/api/ping` 能稳定返回 `status=ok`，二是后端日志能记录请求来源，说明前后端和后端容器自身网络均可正常工作。

Listing 1: 本地容器联调命令

```
docker compose up --build
curl http://localhost:5000/api/ping
curl http://localhost:5000/api/visit
```

预期 `/api/ping` 返回如下 JSON：

```
{"status": "ok"}
```

本任务的验证依据包括：SWR 控制台中 backend/frontend 镜像列表，前端首页中的姓名学号信息，以及后端日志中出现的 GET /api/ping 请求记录。

镜像名称	所属组织	类型	版本号	更新时间	操作
frontend	czk-course	私有	1	2026/05/19 09:18:45 GMT+08:00	镜像自动同步
backend	czk-course	私有	1	2026/05/19 09:18:27 GMT+08:00	镜像自动同步

图 1 SWR 控制台中 backend 与 frontend 镜像列表

4.2 任务 2：CCE 集群搭建

集群部分在华为云 CCE 控制台完成。实验中创建了 2 个 Worker 节点的 Kubernetes 集群，Master 节点由华为云托管。初次使用 kubectl 时，kubeconfig 曾指向本机 127.0.0.1 临时端口，导致 kubectl get nodes 无法连接 API Server；随后更换为 CCE 控制台下载的公网 kubeconfig，并在 PowerShell 中设置 KUBECONFIG 环境变量，重新验证集群节点。

Listing 2: CCE 节点状态验证

```
kubectl get nodes -o wide
```

验证结果显示两个 Worker 节点 IP 分别为 192.168.0.23 和 192.168.0.59，状态均为 Ready，Kubernetes 版本为 v1.34.3-r0-34.0.12.3，高于任务要求的 1.27。至此，后续 Deployment、Service、PVC 和 HPA 实验具备运行基础。

节点验证不仅检查 Ready 状态，也需要关注 VERSION、OS-IMAGE 和 CONTAINER-RUNTIME 等信息。本实验节点操作系统为 Ubuntu 22.04.5 LTS，容器运行时为 containerd，说明集群采用当前主流的 Kubernetes 运行方式。由于后续 Redis PVC、LoadBalancer 和 HPA 都依赖 CCE 的云插件和节点资源，节点状态截图是后续所有任务的基础证据。

```
user@l8nv0e0jhd3urs-machine:~$ kubectl get nodes -o wide
NAME              STATUS    ROLES    AGE   VERSION          INTERNAL-IP    EXTERNAL-IP    OS-IMAGE          KERNEL-VERSION    CONTAINER-RUNTIME
192.168.0.23      Ready     <none>    18m   v1.34.3-r0-34.0.12.3  192.168.0.23   <none>         Ubuntu 22.04.5 LTS  5.15.0-164-generic containerd://1.7.29-11-g0a540c3d5
192.168.0.59      Ready     <none>    3m10s v1.34.3-r0-34.0.12.3  192.168.0.59   <none>         Ubuntu 22.04.5 LTS  5.15.0-164-generic containerd://1.7.29-11-g0a540c3d5
```

图 2 CCE 节点状态验证

```
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl get nodes -o wide
NAME              STATUS    ROLES    AGE   VERSION          INTERNAL-IP    EXTERNAL-IP    OS-IMAGE          KERNEL-VERSION    CONTAINER-RUNTIME
192.168.0.23      Ready     <none>    20h   v1.34.3-r0-34.0.12.3  192.168.0.23   <none>         Ubuntu 22.04.5 LTS  5.15.0-164-generic containerd://1.7.29-11-g0a540c3d5
192.168.0.59      Ready     <none>    20h   v1.34.3-r0-34.0.12.3  192.168.0.59   <none>         Ubuntu 22.04.5 LTS  5.15.0-164-generic containerd://1.7.29-11-g0a540c3d5
```

图 3 本地终端验证 CCE Worker 节点 Ready

4.3 任务 3：应用部署

应用部署阶段将任务 1 中推送到 SWR 的镜像部署到 CCE 集群。实验中先创建 Redis 密码 Secret 和 Redis 地址 ConfigMap，再创建 Redis PVC、Redis Deployment、后端 Deployment 和 Service。后端镜像使用 `swr.cn-south-1.myhuaweicloud.com/czk-course/backend:v1`，前端镜像使用 `swr.cn-south-1.myhuaweicloud.com/czk-course/frontend:v1`。

后端 Deployment 的副本数设置为 2，容器端口为 5000，并配置 CPU、内存的 requests 和 limits；Redis Deployment 的副本数为 1，内存限制不超过 512Mi。Redis 主机名和端口通过 ConfigMap 注入，Redis 密码通过 Secret 注入，避免在 Deployment 中直接写明密码。部署过程中，后端 Pod 曾因为 SWR 私有镜像拉取权限不足出现 401 Unauthorized，解决方法是在集群中创建 `swr-secret`，并在 backend/frontend Deployment 中配置 `imagePullSecrets`。

Listing 3: 应用部署命令

```
kubectl apply -f secret.yaml
kubectl apply -f configmap.yaml
kubectl apply -f redis-pvc.yaml
kubectl apply -f redis-deployment.yaml
kubectl apply -f backend-deployment.yaml
kubectl apply -f frontend-configmap.yaml
kubectl apply -f frontend-deployment.yaml
kubectl apply -f services.yaml
```

部署完成后，使用如下命令验证 Pod 状态和公网访问：

```
kubectl get pods
kubectl get svc backend-svc
curl http://<ELB_IP>/api/ping
```

后端 Service 最初只配置 `kubernetes.io/elb.class: union` 时，EXTERNAL-IP 长时间为 `<pending>`。查看 Service 事件后发现，华为云 CCE 自动创建公网 ELB 还需要 `kubernetes.io/elb.autocreate` 注解。因此实验中补充自动创建公网 ELB 的注解，删除并重建 backend-svc，最终获得公网 IP 110.41.67.203。通过 PowerShell 访问 `http://110.41.67.203/api/ping`，返回 HTTP 200 和 `{"status": "ok"}`，后端日志同步显示 GET /api/ping 请求，说明后端已通过 ELB 正常暴露。

在资源组织上，后端 Service 采用 LoadBalancer 类型用于公网访问，Redis Service 采用 ClusterIP 类型，仅允许集群内部访问。这样的设计符合最小暴露原则：外部用户只需要访问后端 API，不需要直接访问 Redis 数据库。后端通过 ConfigMap 获取 `REDIS_HOST=redis-svc` 和 `REDIS_PORT=6379`，通过 Secret 获取 Redis 密码，从而把业务

镜像和运行环境配置解耦。截图中 backend 两个 Pod 分布在不同节点上，Redis 一个 Pod 独立运行，能够体现 Deployment 副本和 Service 访问入口均已生效。

```
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl get pods -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED	NODE	READINESS
GATES									
backend-857bc7464b-jnpz8	1/1	Running	0	8m27s	10.244.0.11	192.168.0.23	<none>		<none>
backend-857bc7464b-td4gv	1/1	Running	0	8m11s	10.244.0.139	192.168.0.59	<none>		<none>
redis-7cb6d76c74-x4ksn	1/1	Running	0	10m	10.244.0.10	192.168.0.23	<none>		<none>

图 4 后端与 Redis Pod 均处于 Running 状态

```
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl get svc backend-svc redis-svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
backend-svc	LoadBalancer	10.96.185.34	110.41.67.203, 192.168.0.18	80:31072/TCP	6m8s
redis-svc	ClusterIP	10.96.163.10	<none>	6379/TCP	16m

图 5 后端 LoadBalancer Service 与 Redis ClusterIP Service

```
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> curl http://110.41.67.203/api/ping
```

安全警告：脚本执行风险
Invoke-WebRequest 可解析网页内容。解析网页时，可能会运行网页中的脚本代码。
建议的操作：
使用 -UseBasicParsing 开关来避免执行脚本代码。
是否要继续？
[Y] 是(Y) [A] 全是(A) [N] 否(N) [L] 全否(L) [S] 暂停(S) [?] 帮助（默认值为“N”）: A

StatusCode	: 200
StatusDescription	: OK
Content	: {"status":"ok"}
RawContent	: HTTP/1.1 200 OK Connection: keep-alive Keep-Alive: timeout=4 Proxy-Connection: keep-alive Content-Length: 16 Content-Type: application/json Date: Tue, 19 May 2026 03:29:52 GMT Server: Werkzeug/...
Forms	: {}
Headers	: {[Connection, keep-alive], [Keep-Alive, timeout=4], [Proxy-Connection, keep-alive], [Content-Length, 16]...}
Images	: {}
InputFields	: {}
Links	: {}
ParsedHtml	: mshtml.HTMLDocumentClass
RawContentLength	: 16

图 6 通过 ELB 公网 IP 访问 /api/ping 返回 200

```
Windows PowerShell
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl logs deploy/backend --tail=30
Found 2 pods, using pod/backend-857bc7464b-jnpz8
2026-05-19 03:28:39,771 INFO GET /api/ping from 10.244.0.1
2026-05-19 03:28:39,771 INFO 10.244.0.1 - - [19/May/2026 03:28:39] "GET /api/ping HTTP/1.1" 200 -
2026-05-19 03:28:40,048 INFO 10.244.0.1 - - [19/May/2026 03:28:40] "GET /healthz HTTP/1.1" 200 -
2026-05-19 03:28:50,048 INFO 10.244.0.1 - - [19/May/2026 03:28:50] "GET /healthz HTTP/1.1" 200 -
2026-05-19 03:28:54,771 INFO GET /api/ping from 10.244.0.1
2026-05-19 03:28:54,771 INFO 10.244.0.1 - - [19/May/2026 03:28:54] "GET /api/ping HTTP/1.1" 200 -
2026-05-19 03:29:00,048 INFO 10.244.0.1 - - [19/May/2026 03:29:00] "GET /healthz HTTP/1.1" 200 -
2026-05-19 03:29:09,771 INFO GET /api/ping from 10.244.0.1
2026-05-19 03:29:09,771 INFO 10.244.0.1 - - [19/May/2026 03:29:09] "GET /api/ping HTTP/1.1" 200 -
2026-05-19 03:29:10,048 INFO 10.244.0.1 - - [19/May/2026 03:29:10] "GET /healthz HTTP/1.1" 200 -
2026-05-19 03:29:20,048 INFO 10.244.0.1 - - [19/May/2026 03:29:20] "GET /healthz HTTP/1.1" 200 -
2026-05-19 03:29:24,771 INFO GET /api/ping from 10.244.0.1
2026-05-19 03:29:24,771 INFO 10.244.0.1 - - [19/May/2026 03:29:24] "GET /api/ping HTTP/1.1" 200 -
2026-05-19 03:29:30,048 INFO 10.244.0.1 - - [19/May/2026 03:29:30] "GET /healthz HTTP/1.1" 200 -
2026-05-19 03:29:39,771 INFO GET /api/ping from 10.244.0.1
2026-05-19 03:29:39,771 INFO 10.244.0.1 - - [19/May/2026 03:29:39] "GET /api/ping HTTP/1.1" 200 -
2026-05-19 03:29:40,048 INFO 10.244.0.1 - - [19/May/2026 03:29:40] "GET /healthz HTTP/1.1" 200 -
2026-05-19 03:29:50,048 INFO 10.244.0.1 - - [19/May/2026 03:29:50] "GET /healthz HTTP/1.1" 200 -
2026-05-19 03:29:52,451 INFO GET /api/ping from 10.244.0.1
2026-05-19 03:29:52,451 INFO 10.244.0.1 - - [19/May/2026 03:29:52] "GET /api/ping HTTP/1.1" 200 -
2026-05-19 03:29:54,772 INFO GET /api/ping from 10.244.0.1
2026-05-19 03:29:54,772 INFO 10.244.0.1 - - [19/May/2026 03:29:54] "GET /api/ping HTTP/1.1" 200 -
2026-05-19 03:30:00,048 INFO 10.244.0.1 - - [19/May/2026 03:30:00] "GET /healthz HTTP/1.1" 200 -
2026-05-19 03:30:09,771 INFO GET /api/ping from 10.244.0.1
2026-05-19 03:30:09,771 INFO 10.244.0.1 - - [19/May/2026 03:30:09] "GET /api/ping HTTP/1.1" 200 -
2026-05-19 03:30:10,048 INFO 10.244.0.1 - - [19/May/2026 03:30:10] "GET /healthz HTTP/1.1" 200 -
2026-05-19 03:30:20,048 INFO 10.244.0.1 - - [19/May/2026 03:30:20] "GET /healthz HTTP/1.1" 200 -
2026-05-19 03:30:24,771 INFO GET /api/ping from 10.244.0.1
2026-05-19 03:30:24,771 INFO 10.244.0.1 - - [19/May/2026 03:30:24] "GET /api/ping HTTP/1.1" 200 -
2026-05-19 03:30:30,048 INFO 10.244.0.1 - - [19/May/2026 03:30:30] "GET /healthz HTTP/1.1" 200 -
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s>
```

图 7 后端日志显示收到 /api/ping 请求

4.4 任务 4: Redis 持久化存储

Redis 持久化部分使用华为云 EVS 云硬盘。实验中先创建 redis-data-pvc，设置 storageClassName 为 csi-disk，申请容量为 10Gi；随后在 Redis Deployment 中将该 PVC 挂载到容器 /data 目录，并启动 Redis AOF 持久化。这样 Redis 的数据文件不会随着 Pod 删除而丢失。

Listing 4: Redis PVC 验证命令

```
kubectl get pvc
kubectl exec -it <redis-pod-name> -- redis-cli -a <password> SET testkey "hello"
kubectl exec -it <redis-pod-name> -- redis-cli -a <password> GET testkey
kubectl delete pod <redis-pod-name>
kubectl exec -it <new-redis-pod-name> -- redis-cli -a <password> GET testkey
```

验证时先执行 `kubectl get pvc`，确认 `redis-data-pvc` 状态为 `Bound`，`StorageClass` 为 `csi-disk`。随后进入 Redis Pod，执行 `SET testkey "hello"` 写入测试数据，再执行 `GET testkey` 得到 `"hello"`。接着删除当前 Redis Pod，由 Deployment 自动重建新 Pod。新 Pod 运行后再次执行 `GET testkey`，仍返回 `"hello"`，证明数据保存在 PVC 对应的云硬盘中，而不是保存在旧 Pod 的临时文件系统中。

这个实验能区分“Pod 重建”和“数据重建”的区别。Pod 是临时运行单元，删除后其容器文件系统会消失；PVC 绑定的是独立云硬盘，生命周期可以长于 Pod。因此只要 Redis 的数据目录挂载到 PVC，并开启 AOF 或 RDB 持久化，数据就不会因为 Pod 名

称变化而丢失。本实验中旧 Redis Pod 被删除后，新 Pod 名称发生变化，但 testkey 仍能读取，说明持久化路径和 PVC 绑定关系正确。

```
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl get pvc
```

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES	STORAGECLASS	VOLUMEATTR
redis-data-pvc	Bound	pvc-9904abfd-fbe0-4faa-8e7d-da159f0477ac	10Gi	RWO	csi-disk	<unset>

图 8 Redis PVC 状态为 Bound，StorageClass 为 csi-disk

```
Windows PowerShell
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> $REDIS_POD = kubectl get pod -l app=redis -o jsonpath="{.items[0].metadata.name}"
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl exec -it $REDIS_POD -- redis-cli -a redis2026 SET testkey "hello"
Warning: Using a password with '-a' or '-u' option on the command line interface may not be safe.
OK
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl exec -it $REDIS_POD -- redis-cli -a redis2026 GET testkey
Warning: Using a password with '-a' or '-u' option on the command line interface may not be safe.
"hello"
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s>
```

图 9 Redis 写入并读取 testkey=hello

```
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl delete pod $REDIS_POD
pod "redis-7cb6d76c74-x4ksn" deleted from default namespace
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl get pods -l app=redis -w
NAME READY STATUS RESTARTS AGE
redis-7cb6d76c74-bpzrj 1/1 Running 0 7s
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s>
```

图 10 删除 Redis Pod 后由 Deployment 自动重建

```
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> $NEW_REDIS_POD = kubectl get pod -l app=redis -o jsonpath="{.items[0].metadata.name}"
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl exec -it $NEW_REDIS_POD -- redis-cli -a redis2026 GET testkey
Warning: Using a password with '-a' or '-u' option on the command line interface may not be safe.
"hello"
```

图 11 Redis Pod 重建后仍可读取 testkey=hello

4.5 任务 5: ConfigMap Volume 挂载

本任务将前端 Nginx 反向代理配置改为 ConfigMap Volume 挂载。实验中先创建 frontend-nginx-config ConfigMap，在 data.default.conf 中写入完整 Nginx 配置，其中 upstream 指向后端 Service。随后修改 frontend Deployment，将该 ConfigMap 挂载到容器内 /etc/nginx/conf.d/default.conf，并通过 subPath 指定挂载单个配置文件。

Listing 5: ConfigMap Volume 验证命令

```
kubectl apply -f frontend-configmap.yaml
kubectl rollout restart deployment/frontend
kubectl exec -it <frontend-pod-name> -- cat /etc/nginx/conf.d/default.conf
```

为验证 ConfigMap Volume 是否真正生效，实验中将 upstream 端口从 backend-svc:5000 临时修改为 backend-svc:5001，执行 `kubectl apply` 并滚动重启前端 Deployment。进入新的前端 Pod 后，读取 `/etc/nginx/conf.d/default.conf`，可以看到配置文件中已经变为 `server backend-svc:5001;`。截图完成后，再将端口恢复为 5000 并重新发布前端，避免影响后续访问。Volume 挂载方式适合 Nginx 配置、证书、应用配置文件等文件型配置，程序可以像读取普通文件一样读取配置内容；`envFrom` 更适合 Redis 地址、端口、运行环境等少量键值型配置，应用在启动时从环境变量中读取。

与环境变量相比，Volume 挂载更适合完整配置文件，因为 Nginx 本身就是读取 `/etc/nginx/conf.d/default.conf` 文件启动服务。如果使用环境变量，还需要额外模板渲染或入口脚本才能生成配置文件，复杂度更高。ConfigMap Volume 的另一个优点是配置内容可以直接通过 `kubectl get configmap -o yaml` 查看，也可以通过 `kubectl exec` 进入容器检查最终落盘文件，便于验收和排错。

```
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl apply -f frontend-configmap.yaml
configmap/frontend-nginx-config created
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl apply -f frontend-deployment.yaml
deployment.apps/frontend created
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl get configmap frontend-nginx-config -o
yaml
apiVersion: v1
data:
  default.conf: |
    upstream backend_upstream {
      server backend-svc:5000;
    }

    server {
      listen 80;
      server_name _;

      root /usr/share/nginx/html;
      index index.html;

      location / {
        try_files $uri $uri/ /index.html;
      }

      location /api/ {
        proxy_pass http://backend_upstream/api/;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
      }
    }
kind: ConfigMap
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"v1","data":{"default.conf":"upstream backend_upstream {\n  server backend-svc:5000;\n}\n\nserver
{\n  listen 80;\n  server_name _;\n\n  root /usr/share/nginx/html;\n  index index.html;\n\n  location / {\n
    try_files $uri $uri/ /index.html;\n  }\n\n  location /api/ {\n    proxy_pass http://backend_upstream/api/;\n
    proxy_http_version 1.1;\n    proxy_set_header Host $host;\n    proxy_set_header X-Real-IP $remote_addr;\n
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;\n    proxy_set_header X-Forwarded-Proto $sche
me;\n  }\n}\n"},"kind":"ConfigMap","metadata":{"annotations":{},"name":"frontend-nginx-config","namespace":"default"}}
}
```

图 12 ConfigMap 中保存完整 Nginx 反向代理配置

```
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl get deployment frontend -o yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  annotations:
    deployment.kubernetes.io/revision: "1"
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"apps/v1","kind":"Deployment","metadata":{"annotations":{"labels":{"app":"frontend"},"name":"frontend"},"namespace":"default"},"spec":{"replicas":1,"selector":{"matchLabels":{"app":"frontend"},"template":{"metadata":{"labels":{"app":"frontend"},"spec":{"containers":[{"image":"swr.cn-south-1.myhuaweicloud.com/czk-course/frontend:v1","imagePullPolicy":"IfNotPresent","name":"frontend","ports":[{"containerPort":80}], "resources":{"limits":{"cpu":"200m","memory":"128Mi"},"requests":{"cpu":"50m","memory":"64Mi"},"volumeMounts":[{"mountPath":"/etc/nginx/conf.d/default.conf","name":"nginx-config","subPath":"default.conf"}]}], "imagePullSecrets":[{"name":"swr-secret"}], "volumes":[{"configMap":{"items":[{"key":"default.conf","path":"default.conf"},"name":"frontend-nginx-config","name":"nginx-config"}]}]}}}}
  creationTimestamp: "2026-05-19T06:33:12Z"
  generation: 1
  labels:
    app: frontend
  name: frontend
  namespace: default
  resourceVersion: "129140"
  uid: a4c6fa33-087e-469c-8b93-82140047adbef
spec:
  progressDeadlineSeconds: 600
  replicas: 1
  revisionHistoryLimit: 10
  selector:
    matchLabels:
      app: frontend
  strategy:
    rollingUpdate:
      maxSurge: 25%
      maxUnavailable: 25%
    type: RollingUpdate
  template:
    metadata:
      labels:
        app: frontend
    spec:
      containers:
        - image: swr.cn-south-1.myhuaweicloud.com/czk-course/frontend:v1
          imagePullPolicy: IfNotPresent
          name: frontend
          ports:
            - containerPort: 80
              protocol: TCP
          resources:
            limits:
              cpu: 200m
              memory: 128Mi
            requests:
              cpu: 50m
              memory: 64Mi
          terminationMessagePath: /dev/termination-log
          terminationMessagePolicy: File
          volumeMounts:
            - mountPath: /etc/nginx/conf.d/default.conf
              name: nginx-config
              subPath: default.conf
      dnsPolicy: ClusterFirst
      imagePullSecrets:
        - name: swr-secret
      restartPolicy: Always
      schedulerName: default-scheduler
      securityContext: {}
      terminationGracePeriodSeconds: 30
      volumes:
        - configMap:
            defaultMode: 420
            items:
              - key: default.conf
                path: default.conf
            name: frontend-nginx-config
            name: nginx-config
status:
  conditions:
    - lastTransitionTime: "2026-05-19T06:33:12Z"
      lastUpdateTime: "2026-05-19T06:33:12Z"
      message: Deployment does not have minimum availability.
      reason: MinimumReplicasUnavailable
      status: "False"
      type: Available
    - lastTransitionTime: "2026-05-19T06:33:12Z"
      lastUpdateTime: "2026-05-19T06:33:12Z"
      message: ReplicaSet "frontend-7948554c65" is progressing.
      reason: ReplicaSetUpdated
      status: "True"
      type: Progressing
  observedGeneration: 1
  replicas: 1
  unavailableReplicas: 1
  updatedReplicas: 1
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> |
```

图 13 前端 Deployment 通过 Volume 挂载 ConfigMap

```
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl apply -f frontend-deployment.yaml
deployment.apps/frontend configured
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl rollout status deployment/frontend
deployment "frontend" successfully rolled out
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl get pods -l app=frontend -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE               NOMINATED NODE   READINESS GATES
frontend-865c67856f-pgct5           1/1     Running   0           13s   10.244.0.13     192.168.0.23      <none>            <none>
>
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> $FRONTEND_POD = kubectl get pod -l app=frontend -o jsonpath="{.items[0].metadata.name}"
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl exec -it $FRONTEND_POD -- cat /etc/nginx/conf.d/default.conf
upstream backend_upstream {
    server backend-svc:5001;
}

server {
    listen 80;
    server_name _;

    root /usr/share/nginx/html;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }

    location /api/ {
        proxy_pass http://backend_upstream/api/;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

图 14 修改 ConfigMap 端口后在前端 Pod 内验证配置文件已更新

4.6 任务 6：HPA 弹性伸缩

HPA 实验用于验证后端服务能否根据 CPU 利用率自动扩缩容。实验中先安装并检查 metrics-server，使 kubectl top nodes 和 kubectl top pods 能返回 CPU、内存指标；然后创建 backend-hpa，设置 minReplicas=1、maxReplicas=4、CPU 目标利用率为 60%。创建 HPA 后，先将 backend 缩到 1 个副本，便于观察后续扩容过程。

Listing 6: HPA 验证命令

```
kubectl top nodes
kubectl apply -f hpa.yaml
kubectl get hpa
kubectl get pods -w
```

本机没有安装 ab 工具，因此实验中使用 PowerShell Start-Job 并发脚本替代压测。压测脚本向 ELB 公网地址 <http://110.41.67.203/api/ping> 并发发送请求，同时在另一个终端执行 kubectl get pods -l app=backend -w 和 kubectl get hpa 观察副本变化。

```
1..40 | ForEach-Object {
    Start-Job {
        for ($i = 0; $i -lt 500; $i++) {
            Invoke-WebRequest -UseBasicParsing http://110.41.67.203/api/ping | Out-Null
        }
    }
}
```

```
}
}
```

由于 /api/ping 接口逻辑很轻，初次压测时 CPU 利用率没有明显上升。为完成 HPA 验证，实验中临时降低 backend 的 requests.cpu，使 CPU 利用率百分比能反映轻量请求带来的压力。压测期间 HPA 最高观察到 600%/60%，backend 副本数由 1 扩展到 4。停止压测后等待冷却窗口，HPA 最终将副本缩回 1。扩容延迟主要来自 metrics-server 的采集周期和 HPA 控制器的评估间隔；缩容冷却时间用于防止负载短时间波动时频繁扩缩容；HPA 的价值在于高负载时自动增加副本保障可用性，低负载时减少副本降低云资源成本。

HPA 的百分比并不是直接使用节点 CPU 百分比，而是根据 Pod 当前 CPU 使用量与容器 requests.cpu 的比例计算。因此在轻量接口压测场景下，如果 request 设置较大，HPA 可能认为负载仍然很低。实验中先确认 metrics-server 可用，再观察 kubectl get hpa 中 TARGETS 字段变化，最后结合 kubectl get pods -l app=backend -w 记录副本数变化。扩容到 4 个副本后，说明 HPA 控制器、指标采集、Deployment 副本调整三部分链路均已打通；停止压测后缩回 1 个副本，说明缩容策略也能正常工作。

```
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl top nodes
NAME          CPU(cores)   CPU(%)   MEMORY(bytes)  MEMORY(%)
192.168.0.23   75m          3%       995Mi          16%
192.168.0.59   74m          3%       1052Mi         17%
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl top pods
NAME          CPU(cores)   MEMORY(bytes)
backend-857bc7464b-td4gv  1m           33Mi
frontend-5d6b9cb557-blf2z 0m           2Mi
redis-7cb6d76c74-bpzzrj   4m           2Mi
```

图 15 metrics-server 可用，kubectl top nodes/pods 返回资源指标

```
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> 1..40 | ForEach-Object {
>> Start-Job {
>>     for ($i = 0; $i -lt 500; $i++) {
>>         try {
>>             Invoke-WebRequest -UseBasicParsing http://110.41.67.203/api/ping | Out-Null
>>         } catch {}
>>     }
>> }
>> }
```

Id	Name	PSJobTypeName	State	HasMoreData	Location	Command
9	Job9	BackgroundJob	Running	True	localhost	...
11	Job11	BackgroundJob	Running	True	localhost	...
13	Job13	BackgroundJob	Running	True	localhost	...
15	Job15	BackgroundJob	Running	True	localhost	...
17	Job17	BackgroundJob	Running	True	localhost	...
19	Job19	BackgroundJob	Running	True	localhost	...
21	Job21	BackgroundJob	Running	True	localhost	...

图 16 使用 PowerShell 并发请求脚本替代 ab 发起压测

```
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl get hpa
NAME      REFERENCE          TARGETS      MINPODS  MAXPODS  REPLICAS  AGE
backend-hpa  Deployment/backend  cpu: 10%/60%  1        4        1         9m35s
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl top pods
NAME      CPU(cores)  MEMORY(bytes)
backend-857bc7464b-td4gv  1m          33Mi
frontend-5d6b9cb557-blf2z  0m          2Mi
redis-7cb6d76c74-bpzrj  3m          2Mi
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl get pods -l app=backend -w
NAME      READY  STATUS   RESTARTS  AGE
backend-857bc7464b-td4gv  1/1    Running  0         3h41m
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl top pods
NAME      CPU(cores)  MEMORY(bytes)
backend-857bc7464b-td4gv  1m          33Mi
frontend-5d6b9cb557-blf2z  0m          2Mi
redis-7cb6d76c74-bpzrj  3m          2Mi
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl get hpa
NAME      REFERENCE          TARGETS      MINPODS  MAXPODS  REPLICAS  AGE
backend-hpa  Deployment/backend  cpu: 10%/60%  1        4        1         10m
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl get hpa
NAME      REFERENCE          TARGETS      MINPODS  MAXPODS  REPLICAS  AGE
backend-hpa  Deployment/backend  cpu: 600%/60%  1        4        4         15m
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl top pods
NAME      CPU(cores)  MEMORY(bytes)
backend-5c848d6c6d-94xzb  5m          33Mi
backend-5c848d6c6d-c5kpx  5m          32Mi
backend-5c848d6c6d-f6v4w  5m          32Mi
backend-5c848d6c6d-jtxwt  5m          32Mi
frontend-5d6b9cb557-blf2z  0m          2Mi
redis-7cb6d76c74-bpzrj  4m          2Mi
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl get pods -l app=backend -w
NAME      READY  STATUS   RESTARTS  AGE
backend-5c848d6c6d-94xzb  1/1    Running  0         4m1s
backend-5c848d6c6d-c5kpx  1/1    Running  0         4m11s
backend-5c848d6c6d-f6v4w  1/1    Running  0         2m1s
backend-5c848d6c6d-jtxwt  1/1    Running  0         2m1s
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl top pods
NAME      CPU(cores)  MEMORY(bytes)
backend-5c848d6c6d-94xzb  5m          33Mi
backend-5c848d6c6d-c5kpx  5m          32Mi
backend-5c848d6c6d-f6v4w  5m          32Mi
backend-5c848d6c6d-jtxwt  5m          32Mi
frontend-5d6b9cb557-blf2z  0m          2Mi
```

图 17 HPA 在压测期间将 backend 副本扩展到 4 个

```
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl get hpa
NAME      REFERENCE          TARGETS      MINPODS  MAXPODS  REPLICAS  AGE
backend-hpa  Deployment/backend  cpu: 500%/60%  1        4        4         25m
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl get hpa
NAME      REFERENCE          TARGETS      MINPODS  MAXPODS  REPLICAS  AGE
backend-hpa  Deployment/backend  cpu: 100%/60%  1        4        2         26m
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl get hpa
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl get hpa
NAME      REFERENCE          TARGETS      MINPODS  MAXPODS  REPLICAS  AGE
backend-hpa  Deployment/backend  cpu: 100%/60%  1        4        4         27m
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl get hpa
NAME      REFERENCE          TARGETS      MINPODS  MAXPODS  REPLICAS  AGE
backend-hpa  Deployment/backend  cpu: 100%/60%  1        4        4         27m
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl get hpa
NAME      REFERENCE          TARGETS      MINPODS  MAXPODS  REPLICAS  AGE
backend-hpa  Deployment/backend  cpu: 10%/60%  1        4        4         28m
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl get hpa
NAME      REFERENCE          TARGETS      MINPODS  MAXPODS  REPLICAS  AGE
backend-hpa  Deployment/backend  cpu: 10%/60%  1        4        4         29m
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl get hpa
NAME      REFERENCE          TARGETS      MINPODS  MAXPODS  REPLICAS  AGE
backend-hpa  Deployment/backend  cpu: 10%/60%  1        4        1         32m
(base) PS D:\大三下\云计算技术\大作业\cloud-computing-final-project\k8s> kubectl top pods
NAME      CPU(cores)  MEMORY(bytes)
backend-857bc7464b-9lrsj  1m          32Mi
frontend-5d6b9cb557-blf2z  0m          2Mi
redis-7cb6d76c74-bpzrj  3m          2Mi
```

图 18 压测停止后 HPA 缩容回 1 个副本

五、第二部分：Spark 大数据分析

5.1 A-0 环境部署

本报告第二部分选择方向 A：Spark 大数据分析。教师下发离线资源包后，本实验优先采用课程提供的资源完成环境部署：先将 `spark-operator-2.5.0.tar` 和 `pyspark-v9.tar` 导入本地 Docker，再重新打 Tag 推送到本人华为云 SWR 组织 `czk-course`。最终 CCE 集群拉取的控制器镜像为 `swr.cn-south-1.myhuaweicloud.com/czk-course/spark-operator:2.5.0`，PySpark 作业镜像为 `swr.cn-south-1.myhuaweicloud.com/czk-course/pyspark:v9`。其中 `pyspark:v9` 内部 Spark 版本为 3.4.3，并自带 `/opt/spark/examples/src/main/python/wordcount.py` 示例程序和样例输入文件，适合用于 A-0 环境连通性验证。

Listing 7: 教师离线包镜像导入、推送与作业提交命令

```
docker load -i spark/spark-operator-2.5.0.tar
docker load -i spark/pyspark-v9.tar
docker tag ghcr.io/kubeflow/spark-operator/controller:2.5.0 \
  swr.cn-south-1.myhuaweicloud.com/czk-course/spark-operator:2.5.0
docker tag swr.cn-east-3.myhuaweicloud.com/cloud-course-2025212245/pyspark:v9 \
  swr.cn-south-1.myhuaweicloud.com/czk-course/pyspark:v9
docker push swr.cn-south-1.myhuaweicloud.com/czk-course/spark-operator:2.5.0
docker push swr.cn-south-1.myhuaweicloud.com/czk-course/pyspark:v9

helm upgrade --install spark-op ./spark-operator/ \
  -n spark-operator --create-namespace \
  -f spark-operator-teacher-values.yaml

kubectl apply -f sparkapplication-wordcount-teacher.yaml
kubectl get pods -n default
kubectl logs <driver-pod-name>
```

`sparkapplication-wordcount-teacher.yaml` 采用 `cluster` 模式运行 Python 作业，核心参数为 `image=swr.cn-south-1.myhuaweicloud.com/czk-course/pyspark:v9`、`sparkVersion=3.4.3`、`mainApplicationFile=local:///opt/spark/examples/src/main/python/wordcount.py`、`executor.instances=2`、`executor.memory=1g`。Driver Pod 使用 `spark-op-spark-operator-spark` 这个 ServiceAccount 提交作业，两个 Executor 分别由 Spark Operator 自动创建。由于 SWR 镜像仓库未设置为公开，本实验在 `spark-operator` 和 `default` 命名空间创建 `swr-secret`，并在 Operator Helm values 与 SparkApplication 中配置 `imagePullSecrets`，保证 CCE 节点能够认证拉取课程镜像。

实际部署过程中主要处理了三个问题。第一，课程镜像体积较大，推送

spark-operator:2.5.0 和 pyspark:v9 用时较长, 需要确认 Docker 登录凭证有效后再 push。第二, Operator 切换到本人 SWR 镜像后曾出现 401 Unauthorized, 原因是 CCE 节点匿名拉取私有镜像失败, 创建 swr-secret 后问题解决, 控制器 Pod 成功进入 Running, 镜像 ID 指向本人 SWR 中的 spark-operator:2.5.0。第三, 课程 CCE 集群资源较紧, 最初 Driver Pod 曾因 Insufficient cpu 处于 Pending, 因此在 YAML 中设置 spark.kubernetes.driver.request.cores=10m 和 spark.kubernetes.executor.request.cores=10m。最终 pyspark-wordcount-teacher 作业创建了 1 个 Driver Pod 和 2 个 Executor Pod, 两个 Executor 分别调度到两个 Worker 节点运行; 作业结束后 SparkApplication 状态为 COMPLETED, Driver 日志中出现 WordCount 统计结果和 exitCode 0, 说明教师离线资源包版本验证通过。

5.2 A-1 数据清洗

Spark 分析脚本首先读取 OBS 或本地样例数据, 打印 Schema 和前 5 行, 然后统计每个字段的缺失值比例。对于核心指标 rating, 采用 dropna 删除缺失评分记录, 因为评分是后续排序和聚合分析的核心字段; 对于 genre、country、vote_count 等辅助字段, 采用 fillna 填充策略, 以保留更多样本用于总体统计。

Listing 8: 缺失值处理代码

```
no_missing_rating = prepared.dropna(subset=["rating"])
cleaned = no_missing_rating.fillna({
    "genre": "Unknown",
    "country": "Unknown",
    "vote_count": 0,
})
```

清洗阶段需要截图保存 Schema、前 5 行、缺失值比例、清洗前后行数和基本统计信息。当前代码层面已完成两种缺失值处理策略: 核心评分字段采用删除策略, 辅助字段采用填充策略; 运行截图待 Spark Operator 环境完成后补充。

清洗策略的选择依据是字段对后续分析的重要程度。评分字段 rating 直接参与平均分、Top-N 排序和窗口排名, 如果缺失后强行填充, 可能引入人为偏差, 因此采用删除缺失评分记录的方式; 类型、国家和投票数等字段更多用于分组统计和辅助解释, 采用默认值填充可以保留更多样本, 避免因为非核心字段缺失导致整行数据被删除。正式运行时将输出清洗前总行数、删除评分缺失后的行数、填充后的行数, 并通过这些数量变化说明清洗策略对样本规模的影响。

5.3 A-2 Spark SQL 统计分析

本实验设计了五类查询，其中前四类覆盖任务书要求的 GROUP BY 聚合、ORDER BY Top-N、时间维度趋势分析、JOIN 或窗口函数。

表 4 Spark SQL 分析任务

查询类型	分析内容
GROUP BY 聚合	按电影类型统计数量和平均评分，观察不同类型的整体评分差异。
Top-N 排序	在投票数达到阈值的影片中，按评分和投票数选出前 10 名。
时间趋势	按年份统计电影数量和平均评分，分析不同年份评分变化趋势。
窗口函数	在每个类型内部使用 row_number 选出评分前三影片。
JOIN 分析	将细粒度类型映射为更粗粒度分组后进行聚合统计。

每个查询结果都应在报告中附截图，并补充不少于 50 字的分析说明。例如，若某一类型平均评分较高但样本数量较少，应说明其统计稳定性可能低于样本量较大的类型；若 Top-N 结果集中于较新年份，也可结合投票数和数据集分布解释原因。当前 SQL 设计已覆盖评分表要求，结果截图待 Spark 作业实际运行后补齐。

四类核心查询对应不同分析角度：GROUP BY 用于观察总体分布，Top-N 用于寻找高评分高热度样本，时间维度查询用于观察年份变化趋势，窗口函数或 JOIN 查询用于在分组内部做进一步比较。与单纯输出表格相比，报告分析时还需要解释统计结果的可靠性，例如样本量过小的分组平均分可能不稳定，投票数较高的影片更能代表大众评价，年份趋势也可能受到数据集收录范围影响。这样可以避免只展示 SQL 结果而缺少分析。

5.4 A-3 性能对比与 Amdahl 分析

性能对比选取同一个查询，分别使用 Pandas 单机、PySpark 1 executor 和 PySpark 2 executors 实现，并记录执行时间。每种模式建议运行 3 次取平均值。表5为报告记录模板，正式提交时应替换为实际运行耗时。

Amdahl 定律公式为：

$$S(p) = \frac{1}{(1 - f) + \frac{f}{p}} \quad (1)$$

其中 $S(p)$ 表示使用 p 个并行单元时的理论加速比， f 表示程序中可并行部分比例。实际加速比通常低于线性加速，原因包括 Spark 任务启动开销、Executor 调度、数据序列化、Shuffle、网络通信以及不可并行部分。数据量较小时，Spark 的调度成本可能抵消一部分并行收益；数据量增大后，并行计算优势会更加明显。

表 5 性能对比记录表

执行模式	第 1 次/s	第 2 次/s	第 3 次/s
Pandas	待测	待测	待测
PySpark 1 executor	待测	待测	待测
PySpark 2 executors	待测	待测	待测

性能对比阶段计划使用相同输入数据和相同统计逻辑，分别记录 Pandas、PySpark 1 executor、PySpark 2 executors 三种模式的耗时。若记 Pandas 平均耗时为 T_{pandas} ，PySpark 2 executor 平均耗时为 T_{spark2} ，则实际加速比可写为 $S = T_{pandas}/T_{spark2}$ 。再结合 Amdahl 定律，可以反推并行部分比例 f ，并分析为什么实际加速比通常小于 2。主要原因包括 Spark 作业启动时间、Driver 与 Executor 通信、任务切分、Shuffle、序列化与反序列化等开销。对于课程实验规模的数据，Spark 不一定总是比 Pandas 快，但它展示了数据规模继续增大后横向扩展的能力。

六、关键文件与代码说明

本实验大作业工程位于 `cloud-computing-final-project/`。主要文件说明如下：

表 6 工程文件说明

文件或目录	作用
<code>backend/app.py</code>	Flask 后端 API，包含 <code>/api/ping</code> 和 Redis 访问接口。
<code>frontend/static/index.html</code>	Nginx 静态首页，展示验收身份信息。
<code>docker-compose.yml</code>	本地启动 backend、frontend 和 Redis 三个服务。
<code>k8s/*.yaml</code>	CCE 部署所需 Kubernetes 资源清单。
<code>spark/analysis.py</code>	PySpark 数据清洗与 SQL 统计分析代码。
<code>report/screenshot-checklist.md</code>	截图证据清单。

七、总结与收获

本次课程设计将容器镜像、Kubernetes 编排、云负载均衡、持久化存储、配置分离、弹性伸缩和 Spark 并行计算串联成一个完整实践流程。第一部分实际完成了 2 个镜像的

构建与推送, SWR 中 backend/frontend 均使用 v1 Tag; CCE 集群包含 2 个 Worker 节点, 节点状态均为 Ready, Kubernetes 版本为 v1.34.3-r0-34.0.12.3, 高于任务要求的 1.27。应用部署后, 后端 Deployment 最终以 2 副本运行, Redis 以 1 副本运行, 后端 LoadBalancer 获得公网 IP 110.41.67.203, 访问 /api/ping 返回 HTTP 200 和 {"status":"ok"}, 说明本地构建的服务已经完成从本机到云端的迁移。

从工程角度看, 本实验最大的收获不是单条命令本身, 而是对云原生资源之间关系的理解。Deployment 负责维持期望副本数, Service 负责服务发现和访问入口, ConfigMap 与 Secret 将非敏感配置和敏感配置分离, PVC 解决 Redis 这类有状态组件的数据持久化, HPA 则在指标系统可用后根据 CPU 利用率自动调整副本数。Redis 持久化验证中, PVC 容量为 10Gi, StorageClass 为 csi-disk, 删除 Redis Pod 后 testkey 仍返回 hello, 说明数据不依赖 Pod 生命周期。ConfigMap Volume 验证中, Nginx 配置文件修改为 backend-svc:5001 后能在容器内直接看到更新, 截图完成后再恢复为 5000, 体现了文件型配置与环境变量配置的适用差异。

实际排错过程也加深了对云平台运行机制的认识。Docker Desktop 崩溃、SWR manifest 解析失败、私有镜像拉取 401、LoadBalancer 长时间 pending、Pod 因 Insufficient cpu 无法调度、metrics-server Pending、HPA 初始指标为 <unknown> 等问题, 都不是单纯代码错误, 而是本地环境、镜像仓库、云厂商注解、资源请求和监控链路共同作用的结果。通过重新构建镜像、创建 swr-secret、补充 elb.autocreate 注解、降低轻量服务的 CPU request、启用并修正 metrics-server 资源配置, 最终使系统恢复到可验收状态。HPA 压测中, 由于本机没有 ab, 改用 PowerShell 并发请求脚本; backend 副本从 1 个扩展到 4 个, CPU 指标最高达到约 600%/60%, 停止压测后经过冷却窗口缩回 1 个, 说明扩缩容链路已经生效。

第二部分 Spark 方向已经完成 A-0 环境验证, 并最终切换为教师离线资源包方案。实验将 spark-operator-2.5.0.tar 和 pyspark-v9.tar 导入本地 Docker 后推送到本人 SWR, CCE 实际拉取的镜像分别为 czk-course/spark-operator:2.5.0 和 czk-course/pyspark:v9; 其中 PySpark 镜像内 Spark 版本为 3.4.3。部署过程中曾遇到 SWR 私有镜像拉取 401 Unauthorized, 通过创建 swr-secret 并配置 imagePullSecrets 解决; 提交作业时还结合课程集群资源情况降低 Driver 和 Executor 的 CPU request, 避免调度失败。最终 pyspark-wordcount-teacher 作业进入 COMPLETED, 运行过程中能够看到 1 个 Driver Pod 和 2 个 Executor Pod, Driver 日志中包含 WordCount 统计结果和 exitCode 0。该部分让我认识到, 大数据分析任务不仅关注 SQL 结果, 还需要关注运行环境、镜像依赖、Executor 数量、内存配置和数据位置。Amdahl 定律也提醒并行计算不一定线性加速, 实际性能会受到调度开销、Shuffle、序列化、网络通信和不可并行部分影响。总体来说, 本实验从应用交付、平台部署、资源调度、数据持久化、弹性伸缩和并行计算多个层面展示了云计算技术的工程体系, 也让我更清楚地理解

了“能运行、能验证、能解释问题”比单纯写出 YAML 更重要。

八、评分要求对照

8.1 第一部分评分对照

表 7 第一部分任务完成情况对照

任务	关键评分点	分值
任务 1	Dockerfile 保留多阶段构建，后端增加 requests 自选包，前端首页包含姓名与学号；SWR 控制台截图包含镜像名称和 Tag；Docker Compose 本地联调需含后端日志。	10
任务 2	kubectl get nodes -o wide 显示不少于 2 个 Worker 节点为 Ready，且 Kubernetes 版本不低于 1.27。	8
任务 3	后端 Deployment 副本数为 2，含 resources；Redis 副本数为 1；后端 Service 为 LoadBalancer 且可访问 /api/ping；Redis Service 为 ClusterIP；ConfigMap 与 Secret 分离配置。	12
任务 4	Redis PVC 状态为 Bound；删除 Redis Pod 后查询 testkey 仍返回 hello。	10
任务 5	Nginx 配置以 ConfigMap Volume 方式挂载，并通过进入前端 Pod 查看配置文件证明更新生效；报告说明 Volume 与 envFrom 差异。	5
任务 6	HPA 压测时后端 Pod 数由 1 增加到多个，停压测后缩回；报告分析扩容延迟、冷却时间与降本价值。	5

8.2 第二部分评分对照

8.3 报告质量要求

附录 A 核心命令清单

```
docker compose up --build
docker login -u cn-north-4@<AK> -p <SK> swr.cn-north-4.myhuaweicloud.com
kubectl get nodes -o wide
kubectl get pods
kubectl get pvc
kubectl get hpa
helm install spark-op ./spark-operator-chart/ -n spark-operator --create-namespace
```

表 8 第二部分方向 A 评分要求

任务	关键评分点	分值
A-0 环境	Spark Operator 安装成功, Driver 与 Executor Pod 截图完整, SparkApplication 参数正确。	10
A-1 清洗	至少 2 种缺失值处理策略, 输出清洗前后行数对比。	10
A-2 SQL	至少 4 个查询, 覆盖 GROUP BY、Top-N、时间维度、JOIN 或窗口函数; 每个查询附截图和分析。	15
A-3 性能	Pandas 与 PySpark 性能对比图, 结合 Amdahl 定律进行量化分析。	5

表 9 报告质量评分要求

评分项	细则	分值
格式规范	封面包含学号、姓名、班级, 字体统一, 图表有编号和标题。	2
内容完整	任务 1–6 与所选方向任务均有步骤说明, 无空白截图。	4
截图质量	截图清晰, Pod 名称、状态、返回值等关键信息可辨认。	2
分析深度	总结不少于 400 字, 包含量化数据, 不泛泛描述。	2

附录 B 截图清单

报告最终提交前应补齐以下截图: 本地首页、后端日志、SWR 镜像列表、CCE 节点状态、Pod 状态、ELB 访问结果、PVC Bound、Redis 删除 Pod 前后查询结果、ConfigMap Volume 文件内容、HPA 扩缩容过程、Spark Driver/Executor Pod 和 Spark 查询输出。